PIIPS

Precision Intelligent Invoice Processing Suite · v2.2

Architecture, schema, and internals — for whoever maintains this application next.

Architecture

A FastAPI backend, a Vite/React single-page frontend, and a SQL Server database, fronted by IIS as a reverse proxy.

```
Browser (React SPA, frontend/dist)
      |  HTTPS
      v
IIS site "PIIPS_AI"  --- web.config: URL Rewrite -> http://127.0.0.1:8000
      |
      v
Uvicorn (NSSM service "PIIPS_Backend", 127.0.0.1:8000)
      |
      v
FastAPI app (app.py)  ---calls--->  Service First / NAV API (config: sf_api_url)
      |
      v
SQL Server (database.py, pyodbc)  |  Working folder (config_store: folder_path)
                                  |    Input / New_Format / Unsupported /
                                  |    Incomplete Data / ALL_INVOICES / output
```

TERM	MEANING
Purchase Header/Line/Reservation	The three sheets an extracted invoice becomes: one header row, N line rows, N reservation rows.
Template	A per-vendor extraction profile: entity, invoice type, PO number format, static values.
Tracker	tbl_Purchase_Tracker — one row per invoice, carrying its lifecycle status and audit trail.
Lifecycle	LOADED → POSTED → COMPLETE, driven by the Load/Post/Complete screens.

Backend module map

FILE	RESPONSIBILITY
app.py	FastAPI routes: auth, users, mail settings, announcements, config, invoices, publish, manuals.
database.py	All SQL Server access — schema DDL (idempotent, run on every startup/DB-config-save), stored procedures, and the Python wrappers around them.
processor.py / anchor_extract.py / ocr_engine.py / invoice_schema.py	PDF text extraction, template/anchor matching, field parsing.
mailer.py	SMTP sending (stdlib smtplib only) + the four HTML email templates.
config_store.py	config.json read/write — folder_path, db_connection, sf_api_url, publish_root.
secret_store.py	Windows DPAPI encrypt/decrypt (LOCAL_MACHINE scope) for secrets at rest.
deploy.py	The Super Admin Publish feature — git pull/merge, frontend build, robocopy to a local staging folder.

v2.2 note. Column detection in anchor_extract.py was made more robust: the three-column Bill To / Ship To / Invoice Details header is now located structurally even when a vendor's Invoice Details column carries no literal caption, and OCR word-continuation handling in ocr_engine.py was fixed so wrapped or overlapping word boxes stitch back together correctly instead of dropping or misordering text. See CHANGELOG.md for the full list of extraction fixes in this release.

v2.2 note. Default payment-terms fallback (used only when neither Service First nor the PDF states a usable value) changed from 45 to 30 days; the Dashboard Fields drill-down popup dropped its redundant HSN_Type row and renamed 'ProductNo' to 'HSN Number'.

v2.2 note. HSN/SAC Code is now conditionally mandatory for an Item (part) line: missing only when the PDF itself had an HSN that Service First failed to confirm - if the PDF never printed one either, nothing is flagged. Implemented in excel_export._line_missing_fields (the DATA MISMATCH gate) and database.get_invoice_field_check (the Fields popup), both comparing against the PDF's raw HSN reading. The Fields popup's Purchase Header section also now shows Buyer Order No.

Database schema (this release)

Schema changes are idempotent DDL run automatically — on every app startup, and again whenever Database Configuration is saved (so pointing at a brand-new empty database just works).

TABLE	NOTES
tbl_user	+ `Email`, `MustChangePassword` columns this release.
tbl_MailSettings	**New.** SettingID IDENTITY(31,1); single row; Password is DPAPI-encrypted.
tbl_Announcement	**New.** Title, BodyText, ImagePath, VideoUrl, EndDateTime, IsActive, Stopped* audit columns.
tbl_Purchase_Header / Line / Tracker, tbl_Reservation_Entry, tbl_InputFile_Log	Unchanged this release.

PIIPS_LIVE bootstrap. `PIIPS.sql` (git-ignored, local artifact only — it contains a real password hash) drops and recreates a target database from scratch: full current schema plus master data only (user types, statuses, a clean default Sadmin, blank mail-settings password, business templates). Regenerate it from a live database's actual schema rather than hand-editing it if the schema has moved on.

Authentication & password policy

Every password on PIIPS is system-generated (`database.generate_temp_password()`) and emailed — there is no code path where an admin types a password for someone else. Policy (`validate_password_policy`): minimum 5 characters, at least one uppercase, one lowercase, one digit, one of `!@#$%^&*()-_=+[]{};:,.<>?/`. Hashing is PBKDF2-SHA256 (`hash_password` / `verify_password`).

Who can reset whose password

Enforced server-side in `POST /api/users/reset-password` (mirrored client-side by `UserManagement.jsx`'s `canAssignFor`): Super Admin → anyone including themselves; Admin → only plain User/Accounts targets, never themselves, another Admin, or a Super Admin.

The default Sadmin account

`DEFAULT_SUPER_ADMIN_USERNAME = "Sadmin"`, seeded by `ensure_default_super_admin()` on every startup if missing — fixed password, no email, never forced to change it, so it can never be locked out by mail-server misconfiguration. Any number of other Super Admins may also exist, created normally with email + a generated password. Both `POST /api/users/active` (deactivating) and the User Management UI explicitly block anyone from deactivating Sadmin, and block deactivating your own account — added this release after a real self-lockout during testing (a Super Admin's own row had no such guard).

> **v2.2 note.** The Viewer role is the one exception to "no code path where an admin types a password for someone else": a Super Admin creating a Viewer account (POST /api/users with user_type_id for Viewer) supplies the username and password directly - no email is required, no temp password is generated, and validate_password_policy is skipped entirely for this one case (an admin-assigned internal credential, e.g. an employee code, not a self-service one). MustChangePassword is also left 0 (see the new @MustChangePassword param on usp_CreateUser) so that password stays permanent. Activating/deactivating a Viewer sends no email either. See CHANGELOG.md for the full Viewer role write-up, including which menu items it does and does not see.

Email (mailer.py)

stdlib `smtplib` / `email.mime` only — no new dependency. SMTP account lives in `tbl_MailSettings`, edited via the Mail Server Setting screen (Super Admin only), saved only after a live connect+login test succeeds.

Four templates, one shell

`welcome_email_html`, `password_reset_email_html`, `deactivation_email_html`, `activation_email_html` — all share `_shell()`: an indigo-to-magenta gradient header matching the app's logo, a monospace password chip, and the PIIPS mark embedded as a base64 `data:` URI (not a URL reference — a `request.base_url`-derived image URL is only reachable from wherever the request happened to originate, not necessarily from the actual recipient's device/network).

Sign-in links point at the right environment

`_base_url(request)` (app.py) prefers the caller's browser `Origin` / `Referer` header over `request.base_url` — the latter reflects Uvicorn's own view of itself behind the IIS reverse proxy (`http://127.0.0.1:8000`), which is wrong on every environment unless IIS forwards proxy headers Uvicorn is configured to trust (it isn't here). Fixed this release after Live-environment emails were linking back to the internal address.

?signout=1

Every "Sign in to PIIPS" link carries this param. `App.jsx`'s `loadUser()` checks for it, clears any already-active session, and strips it from the URL — a welcome/reset email is often opened in the same browser an admin just used to create the account.

Announcements

Super Admin publishes a site-wide notice (`tbl_Announcement`): text, an optional image (saved under `announcement_media/` , git-ignored and excluded from Publish), an optional video URL, and an end date/time; stoppable early at any point.

`AnnouncementBell` (components.jsx) polls `GET /api/announcements/active` every 15 seconds and also listens for a same-tab `ANNOUNCEMENT_CHANGED_EVENT` DOM event (dispatched right after a publish/stop) so the publisher's own session updates instantly rather than waiting on the next poll. Read state is tracked client-side in `localStorage` (`piips_read_announcements`) — deliberately a bell + popup, not an inline banner, so it never disturbs whatever the user is doing on the Dashboard.

Configuration & secrets

SETTING	WHERE
folder_path, db_connection, sf_api_url, publish_root	config.json (git-ignored, machine-specific)
Mail SMTP account	tbl_MailSettings (DB), Password DPAPI-encrypted

`secret_store.protect()` / `unprotect()` wrap Windows DPAPI, LOCAL_MACHINE scope, marked with an `enc:dpapi:` prefix. `config_store.ensure_secret_encrypted()` runs on every startup and re-encrypts a plaintext `db_connection` in place.

> **Machine-locked, every time.** A DPAPI-encrypted value copied from one machine (config.json's db_connection, or tbl_MailSettings.Password) is **useless** on a different machine — it can only be decrypted where it was encrypted. Always re-enter it through the app's own UI after moving to a new environment; never copy the raw encrypted value across machines.

PIIPS · Developer Manual

Deployment (summary)

See **PIIPS_UAT_Deployment_Guide.pdf** (this manual's companion document, Super Admin only) for the full step-by-step: IIS/ARR reverse-proxy setup, the NSSM service, and the common-problems list. In short:

- **Publish** (Super Admin menu) stages a build locally — git pull/merge the right branch, `npm run build`, robocopy to a staging folder. It does not touch the remote server.
- Moving a staged build onto the real server, and restarting the service there, is still a manual step (robocopy `/MIR` + `Restart-Service PIIPS_Backend`).
- **web.config** must remove the WebDAV module/handler if the server has that IIS role feature installed — otherwise it silently intercepts POST/PUT/DELETE (GET still works), which surfaced this release as "Method Not Allowed" on every save action. Harmless to remove when WebDAV isn't present.